

Maestría en Ciencia de Datos & IA
IA Fundamentals

Hill Climbing y Simulated Annealing aplicados al Problema de la Mochila

Práctica 01

Cerrón Calistro, Cristhina Remigia Julon Chamana, Sebastián
Velazque Tito, Jesús Antonio Pereda Escalante, Axel Alexander

Docente: Ph.D. Vicente Machaca Arceda

Repositorio: <https://github.com/AlexanderPereda/iafundamentals-pc1>

Índice

1. Introducción	2
2. Metodología	3
2.1. Diseño experimental general	3
2.2. Problema 1 — Hill Climbing Simple (0/1 Knapsack clásico)	3
2.3. Problema 2 — Hill Climbing con Reinicios Aleatorios (Knapsack Multidimensional)	4
2.4. Problema 3 — Simulated Annealing Estándar (Bounded Knapsack)	6
2.5. Problema 4 — Simulated Annealing Adaptativo (0/1 Knapsack con Penalización)	8
3. Resultados	9
4. Conclusiones	11

1. Introducción

Descripción del problema

El *Problema de la Mochila* (*Knapsack Problem*) es uno de los problemas clásicos de optimización combinatoria. En su forma más simple, dado un conjunto de n ítems, cada uno con un valor v_i y un peso w_i , se busca seleccionar un subconjunto que maximice el valor total sin superar la capacidad W de la mochila. Formalmente:

$$\text{máx} \sum_{i=1}^n v_i x_i \quad \text{sujeto a} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}$$

El problema pertenece a la clase NP-difícil: aunque existe un algoritmo de programación dinámica con complejidad $O(nW)$, este es pseudo-polinomial y se vuelve intratable cuando n y W crecen. Para $n = 250$ ítems con pesos enteros en $[1, 15]$ y $W \approx 750$, el espacio de soluciones contiene $2^{250} \approx 10^{75}$ candidatos, haciendo imposible la enumeración exhaustiva.

Motivación

La elección del Problema de la Mochila como caso de estudio responde a varias razones:

1. **Relevancia práctica.** Tiene aplicaciones directas en asignación de recursos, selección de proyectos de inversión, carga de contenedores logísticos, empaquetado de datos y planificación de rutas. Cualquier problema donde se deba seleccionar un subconjunto de opciones bajo restricciones de capacidad puede modelarse como una variante del Knapsack.
2. **Escalabilidad controlada.** A diferencia de otros problemas NP-difíciles, el Knapsack permite generar instancias de dificultad creciente simplemente aumentando n , lo que facilita el análisis empírico del comportamiento de los algoritmos.
3. **Riqueza de variantes.** Existen múltiples extensiones naturales (multidimensional, bounded, con penalización) que permiten comparar cómo los algoritmos se adaptan a estructuras de restricciones distintas sin cambiar el marco general del problema.
4. **Idoneidad para metaheurísticas de búsqueda local.** La representación binaria permite definir vecindades simples y eficientes (bit-flip), y la función objetivo es fácil de evaluar incrementalmente, lo que hace del Knapsack un banco de pruebas ideal para Hill Climbing y Simulated Annealing.

Alcance del trabajo

En este trabajo se estudian dos familias de metaheurísticas de búsqueda local aplicadas a cuatro variantes del problema, progresando desde la formulación más simple hacia variantes con mayor complejidad estructural:

- **Hill Climbing Simple** sobre el 0/1 Knapsack clásico.
- **Hill Climbing con Reinicios Aleatorios** sobre el Knapsack Multidimensional (dos restricciones simultáneas: peso y volumen).

- **Simulated Annealing Estándar** sobre el Bounded Knapsack (cada ítem puede incluirse hasta c_i veces).
- **Simulated Annealing con Enfriamiento Adaptativo y Penalización** sobre el 0/1 Knapsack, permitiendo explorar regiones infactibles durante la búsqueda.

El objetivo es comparar el comportamiento de cada algoritmo en términos de calidad de solución, tiempo de ejecución y robustez frente a distintos puntos de inicio aleatorios.

2. Metodología

2.1. Diseño experimental general

Antes de describir cada algoritmo, se explican las decisiones de diseño comunes a todos los experimentos.

Generación de instancias. Cada instancia se genera aleatoriamente con una semilla fija para reproducibilidad. Los valores se muestrean uniformemente en $[1, 30]$ y los pesos en $[1, 15]$. Estos rangos crean instancias heterogéneas (ítems de distinta densidad valor/peso), lo que impide que una heurística greedy trivial encuentre siempre la solución óptima. La capacidad de la mochila se fija en el 40 % del peso total $\sum w_i$, salvo en el Bounded Knapsack donde se usa el 50 %. Elegir el 40 % garantiza que el problema sea *no trivial*: con menos del 30 % de capacidad, el espacio de soluciones factibles es demasiado pequeño; con más del 60 %, casi todos los ítems caben y el problema pierde dificultad.

Escalas evaluadas. Se usaron $n \in \{10, 50, 100, 250\}$ ítems. El valor $n = 10$ permite verificar manualmente los resultados. $n = 50$ y $n = 100$ representan instancias medianas donde se aprecia la diferencia entre algoritmos. $n = 250$ es suficientemente grande para observar el comportamiento asintótico del tiempo de ejecución sin exceder tiempos imprácticamente largos.

Semillas aleatorias. Se usaron tres semillas independientes ($\{7, 42, 123\}$) por configuración. Esto permite estimar la *variabilidad* de cada algoritmo: un algoritmo que produce resultados muy distintos según la semilla es menos robusto que uno que converge a valores similares independientemente del punto de inicio. Con tres corridas se obtiene una estimación básica de media y desviación estándar sin incurrir en un costo computacional excesivo.

2.2. Problema 1 — Hill Climbing Simple (0/1 Knapsack clásico)

Representación de la solución. Cada solución es un vector binario $x \in \{0, 1\}^n$, donde $x_i = 1$ indica que el ítem i está incluido en la mochila. Esta representación es natural para el 0/1 Knapsack porque cada ítem solo puede estar incluido o excluido.

Solución inicial. Se genera aleatoriamente incluyendo cada ítem con probabilidad $p = 0.3$. Esta probabilidad no es arbitraria: se eligió deliberadamente por debajo de 0.5 para que la solución inicial ocupe *aproximadamente el 30 % de la capacidad*, dejando margen para que el algoritmo mejore añadiendo ítems. Si se usara $p = 0.5$, la mitad de los ítems estarían inicialmente incluidos y es probable que la solución ya supere la capacidad W , obligando a eliminar ítems inmediatamente y reduciendo el espacio de exploración efectivo. Si se usara $p \approx 0$, el algoritmo partiría de la mochila vacía, lo que es factible pero introduce un sesgo inicial muy fuerte. Si la solución generada supera W (lo cual puede ocurrir por aleatoriedad), se repara eliminando ítems seleccionados al azar en orden

aleatorio hasta restablecer la factibilidad. Se elige eliminación aleatoria (y no eliminación del ítem de menor valor o mayor peso) para no sesgar la búsqueda hacia ninguna región particular del espacio de soluciones.

Vecindad — bit-flip completo (steepest ascent). En cada iteración se generan los n vecinos invirtiendo un bit a la vez ($x_i : 0 \rightarrow 1$ o $1 \rightarrow 0$). El algoritmo selecciona el vecino de mayor fitness entre todos (*steepest ascent* o ascenso más pronunciado). Esta estrategia garantiza el mayor avance posible en cada paso, a diferencia del *first improvement* que acepta el primer vecino que mejore. Se prefiere steepest ascent porque, dada la sencillez del HC Simple (una sola corrida, sin mecanismos de escape), conviene extraer el máximo beneficio de cada iteración aunque sea más costoso computacionalmente.

Función de fitness.

$$f(x) = \begin{cases} \sum_{i=1}^n v_i x_i & \text{si } \sum_{i=1}^n w_i x_i \leq W \\ 0 & \text{en caso contrario} \end{cases}$$

Las soluciones infactibles reciben fitness 0. Esto las penaliza completamente sin asignarles un valor negativo, evitando que el gradiente de búsqueda guíe al algoritmo hacia zonas infactibles. Dado que la solución inicial siempre es factible y que el algoritmo solo acepta mejoras, en la práctica el HC Simple nunca abandona la región factible.

Criterio de parada. El algoritmo se detiene cuando ninguno de los n vecinos supera el fitness actual, lo que indica que se ha alcanzado un *óptimo local*. No se impone un límite de iteraciones porque el número de iteraciones observado es pequeño (2–34 según las tablas), por lo que el criterio natural de convergencia es suficiente.

La Tabla 1 muestra los resultados de las 12 corridas experimentales ($n \in \{10, 50, 100, 250\}$, semillas $\{7, 42, 123\}$).

Cuadro 1: Resultados completos — Hill Climbing Simple (0/1 Knapsack)

n	Semilla	Valor	Peso usado	Capacidad	Iters.	Tiempo (ms)
10	7	86	22	23	5	0.2164
10	42	87	26	28	4	0.1632
10	123	125	23	23	2	0.1089
50	7	241	154	154	2	1.1296
50	42	437	150	150	8	3.6388
50	123	416	189	189	5	2.1683
100	7	544	313	313	2	3.1702
100	42	717	334	334	10	20.3285
100	123	924	291	291	13	27.9420
250	7	1606	814	814	10	105.9757
250	42	2001	770	770	34	333.0265
250	123	1764	832	832	20	161.8711

2.3. Problema 2 — Hill Climbing con Reinicios Aleatorios (Knapsack Multidimensional)

Representación y vecindad. Idénticas al Problema 1 (vector binario y bit-flip). La diferencia radica en que el problema ahora impone *dos* restricciones de capacidad simultáneas: peso máximo W y volumen máximo V , ambas fijadas al 40 % de la suma total de pesos y volúmenes respectivamente. Añadir la dimensión de volumen modela

escenarios más realistas (por ejemplo, un contenedor que tiene tanto un límite de peso como de espacio físico).

Función de fitness.

$$f(x) = \begin{cases} \sum_{i=1}^n v_i x_i & \text{si } \sum_{i=1}^n w_i x_i \leq W \wedge \sum_{i=1}^n \text{vol}_i x_i \leq V \\ 0 & \text{en caso contrario} \end{cases}$$

Una solución es factible solo si respeta *ambas* restricciones simultáneamente. La reparación de la solución inicial también respeta ambas: si alguna de las dos restricciones se viola, se eliminan ítems al azar hasta que las dos se satisfagan.

Motivación de los reinicios aleatorios. El HC Simple queda atrapado en el primer óptimo local que encuentra. La estrategia de reinicios aleatorios (*Random Restart Hill Climbing*) mitiga este problema ejecutando R corridas independientes desde puntos de inicio distintos y retornando la mejor solución encontrada. Cada reinicio r usa la semilla $\text{seed} + 7r$, donde el desplazamiento de 7 asegura que las semillas estén suficientemente separadas en la secuencia aleatoria para producir soluciones iniciales diversas. A mayor R , mayor cobertura del espacio de búsqueda, pero el tiempo crece linealmente con R . Se evaluaron $R \in \{5, 15, 30\}$ para observar la relación entre número de reinicios, calidad de la solución y tiempo de ejecución.

La Tabla 2 muestra los 36 experimentos realizados ($n \in \{10, 50, 100, 250\}$, semillas $\{7, 42, 123\}$, $R \in \{5, 15, 30\}$).

Cuadro 2: Resultados completos — HC con Reinicios Aleatorios
(Knapsack Multidimensional)

n	Sem.	R	Valor	P.usado	Cap.W	V.usado	Cap.V	Iters.	T.(ms)
10	7	5	73	22	23	13	20	2	0.8623
10	7	15	83	21	23	16	20	3	2.0180
10	7	30	83	21	23	16	20	3	4.7025
10	42	5	61	24	28	22	24	2	0.4711
10	42	15	78	23	28	22	24	4	1.6338
10	42	30	78	23	28	22	24	4	3.5605
10	123	5	98	13	23	22	22	3	0.5706
10	123	15	98	13	23	22	22	3	1.6218
10	123	30	98	13	23	22	22	3	3.4106
50	7	5	339	148	154	111	111	5	11.3515
50	7	15	346	150	154	111	111	4	45.8111
50	7	30	346	150	154	111	111	4	94.6874
50	42	5	393	150	150	109	111	7	18.4573
50	42	15	465	150	150	111	111	15	84.0229
50	42	30	465	150	150	111	111	15	128.0555
50	123	5	414	189	189	96	108	11	17.8309
50	123	15	440	174	189	108	108	10	59.6226
50	123	30	446	189	189	101	108	9	106.3037
100	7	5	672	311	313	222	222	11	69.6590
100	7	15	748	310	313	222	222	13	302.2010
100	7	30	748	310	313	222	222	13	614.9817
100	42	5	856	334	334	210	213	15	96.3770
100	42	15	856	334	334	210	213	15	317.0020
100	42	30	856	334	334	210	213	15	623.6086
100	123	5	768	289	291	229	229	6	69.6136

Cuadro 2 — continuación

n	Sem.	R	Valor	P.usado	Cap. W	V.usado	Cap. V	Iters.	T.(ms)
100	123	15	851	288	291	229	229	13	190.6501
100	123	30	851	288	291	229	229	13	433.1392
250	7	5	1794	814	814	490	520	23	1175.7741
250	7	15	1954	814	814	511	520	41	4242.9314
250	7	30	1954	814	814	511	520	41	8277.3570
250	42	5	1869	770	770	517	549	22	1217.6769
250	42	15	2132	770	770	544	549	37	5104.6134
250	42	30	2132	770	770	544	549	37	9470.6732
250	123	5	1930	829	832	494	494	30	1277.3788
250	123	15	1930	829	832	494	494	30	3944.4797
250	123	30	1930	829	832	494	494	30	7575.1827

2.4. Problema 3 — Simulated Annealing Estándar (Bounded Knapsack)

Representación de la solución. Cada solución es un vector entero $x = (x_1, \dots, x_n)$ con $x_i \in \{0, 1, \dots, c_i\}$, donde $c_i \in [1, 5]$ es la cantidad máxima disponible del ítem i (generada aleatoriamente). Esta representación entera es la natural para el Bounded Knapsack: a diferencia del 0/1, aquí cada ítem puede tomarse múltiples veces hasta su límite c_i . La capacidad de la mochila se fija en el 50 % del peso total máximo ($\sum w_i c_i$) en lugar del 40 % usado en los problemas anteriores. El 50 % se eligió porque al poder repetir ítems, el espacio de soluciones factibles es proporcionalmente mayor, y un umbral del 40 % resultaría en instancias demasiado restrictivas.

Solución inicial. A diferencia de los problemas HC, el SA puede manejar soluciones infactibles mediante el criterio de Metropolis. Por ello, la solución inicial se genera *completamente al azar*: cada x_i se muestrea uniformemente en $\{0, \dots, c_i\}$ sin verificar la factibilidad. Esto amplía el espacio de exploración desde el inicio.

Vecindad. En cada paso se selecciona un ítem i uniformemente al azar y se le aplica un incremento $\delta \in \{-1, +1\}$ elegido al azar, clamping el resultado al rango $[0, c_i]$:

$$x'_i = \max(0, \min(c_i, x_i + \delta))$$

Esta vecindad de *cambio mínimo* genera un único vecino por paso (a diferencia del steepest ascent del HC). Esto es apropiado para SA porque el algoritmo evalúa un único vecino en cada iteración según el criterio de Metropolis, sin necesidad de explorar toda la vecindad.

Función de fitness.

$$f(x) = \begin{cases} \sum_{i=1}^n v_i x_i & \text{si } \sum_{i=1}^n w_i x_i \leq W \\ 0 & \text{en caso contrario} \end{cases}$$

Las soluciones infactibles reciben fitness 0. El SA puede aceptarlas de todas formas si $\Delta f = 0 - f_{\text{actual}} < 0$ con probabilidad $e^{\Delta f/T}$, lo que a temperaturas altas es probable. Esto permite al algoritmo explorar regiones infactibles brevemente y encontrar caminos hacia soluciones factibles que de otra forma serían inaccesibles.

Esquema de enfriamiento geométrico. La temperatura evoluciona como $T_k = T_0 \cdot \alpha^k$, donde k es el número de nivel de temperatura completado. Se fijó $T_0 = 100$: suficientemente alto para que al inicio la probabilidad de aceptar un movimiento de deterioro

sea significativa ($e^{-1/100} \approx 0.99$ para un deterioro de 1 unidad), lo que permite exploración amplia. Se fijó $T_{\min} = 0.01$: cuando la temperatura llega a este valor, $e^{-1/0.01} \approx 0$, es decir, prácticamente no se aceptan movimientos de deterioro y el algoritmo converge. En cada nivel de temperatura se realizan $\text{iter_per_T} = 50$ intentos de movimiento antes de enfriar, lo que permite equilibrar la distribución de Boltzmann en cada nivel. Se evaluaron $\alpha \in \{0.90, 0.95, 0.99\}$ para estudiar el efecto de la velocidad de enfriamiento: $\alpha = 0.90$ enfría rápido (88 niveles), $\alpha = 0.99$ enfría lentamente (917 niveles), explorando más extensamente a costa de mayor tiempo.

La Tabla 3 muestra los 36 experimentos realizados ($n \in \{10, 50, 100, 250\}$, semillas $\{7, 42, 123\}$, $\alpha \in \{0.90, 0.95, 0.99\}$).

Cuadro 3: Resultados completos — SA Estándar (Bounded Knapsack)

n	Sem.	α	Valor	Peso usado	Capacidad	Niveles T	Tiempo (ms)
10	7	0.90	203	97	97	88	24.6118
10	7	0.95	211	94	97	180	70.8858
10	7	0.99	214	95	97	917	285.8543
10	42	0.90	332	122	122	88	21.8593
10	42	0.95	332	116	122	180	44.7767
10	42	0.99	332	119	122	917	214.9572
10	123	0.90	407	92	92	88	23.3802
10	123	0.95	407	92	92	180	42.3835
10	123	0.99	407	92	92	917	214.9666
50	7	0.90	1381	556	559	88	47.6915
50	7	0.95	1509	559	559	180	103.9035
50	7	0.99	1563	559	559	917	506.0181
50	42	0.90	1557	557	557	88	47.5959
50	42	0.95	1650	553	557	180	98.1868
50	42	0.99	1711	556	557	917	499.4475
50	123	0.90	1569	701	701	88	52.4803
50	123	0.95	1603	697	701	180	100.8291
50	123	0.99	1631	701	701	917	502.5832
100	7	0.90	2819	1206	1206	88	83.0192
100	7	0.95	3083	1206	1206	180	169.3803
100	7	0.99	3317	1206	1206	917	849.8745
100	42	0.90	2723	1252	1252	88	82.1586
100	42	0.95	2968	1252	1252	180	161.9611
100	42	0.99	3196	1252	1252	917	859.5619
100	123	0.90	3430	1118	1118	88	78.6222
100	123	0.95	3777	1118	1118	180	159.9079
100	123	0.99	4053	1118	1118	917	857.8950
250	7	0.90	6629	2962	2962	88	181.6129
250	7	0.95	6875	2962	2962	180	365.6864
250	7	0.99	7648	2962	2962	917	1904.5840
250	42	0.90	7039	2908	2908	88	185.0138
250	42	0.95	7526	2908	2908	180	370.8570
250	42	0.99	8456	2908	2908	917	1877.9781
250	123	0.90	6548	2852	2852	88	174.7864
250	123	0.95	6790	2851	2852	180	370.9277
250	123	0.99	7617	2852	2852	917	1842.3842

2.5. Problema 4 — Simulated Annealing Adaptativo (0/1 Knapsack con Penalización)

Representación de la solución. Vector binario $x \in \{0, 1\}^n$, igual que en el Problema 1. Los valores de los ítems se muestrean en $[1, 50]$ (en lugar de $[1, 30]$) y los pesos en $[1, 20]$ (en lugar de $[1, 15]$), generando instancias con mayor densidad valor/peso y mayor variabilidad, lo que hace el problema inherentemente más difícil de resolver de forma greedy.

Vecindad — flip de 1 o 2 bits. En cada iteración se invierten $k \in \{1, 2\}$ bits elegidos aleatoriamente, donde k se selecciona con probabilidad uniforme. Incluir el flip de 2 bits *amplía* la vecindad respecto al flip simple: permite pasar de una solución a otra que difiere en dos posiciones simultáneamente, algo que requeriría dos pasos con el flip de 1 bit. Esto introduce mayor diversificación en la exploración, especialmente útil cuando el SA opera con penalización y el paisaje de fitness es más irregular. Se limita a 2 bits (y no más) para no saltar demasiado lejos y perder la naturaleza de búsqueda local.

Función de fitness con penalización. En lugar de descartar soluciones infactibles asignándoles fitness 0, se les asigna un valor penalizado proporcional al exceso de peso:

$$f(x) = \sum_{i=1}^n v_i x_i - \lambda \cdot \max\left(0, \sum_{i=1}^n w_i x_i - W\right)$$

La motivación es que con fitness 0 para infactibles, el algoritmo percibe todas las soluciones infactibles como igualmente malas, perdiendo información sobre cuáles están “más cerca” de la región factible. Con penalización proporcional, el gradiente de búsqueda puede guiar al algoritmo *a través* de la región infactible hacia soluciones factibles de mejor calidad. El coeficiente λ controla el equilibrio entre explorar valor y respetar la restricción:

- $\lambda = 1$: penalización débil. El algoritmo prioriza el valor y puede quedarse en regiones infactibles durante mucho tiempo, sin encontrar nunca una solución factible. Esto explica los valores 0 observados en la Tabla 4 para $n \geq 50$.
- $\lambda = 5$: penalización moderada. Equilibrio adecuado entre exploración y factibilidad; produce los mejores resultados en la mayoría de las instancias.
- $\lambda = 10$: penalización fuerte. El algoritmo se mantiene más cerca de la región factible, pero puede perder soluciones de alto valor que requieren pasar brevemente por zonas infactibles.

Crucialmente, la solución reportada es siempre la mejor **factible** encontrada *a lo largo de toda la búsqueda*, no la solución final. Esto es necesario porque la función penalizada puede terminar en un punto infactible.

Esquema de enfriamiento adaptativo de Lundy & Mees. A diferencia del enfriamiento geométrico (que reduce T por un factor fijo α), el esquema de Lundy & Mees actualiza la temperatura en cada iteración individual:

$$T_{k+1} = \frac{T_k}{1 + \beta \cdot T_k}$$

Este esquema tiene la propiedad de que el enfriamiento es *más lento* cuando la temperatura es baja (la temperatura decrece menos por iteración a medida que se enfría), lo que permite una exploración más fina cerca de la convergencia. Se fijó $\beta = 0.001$: con $T_0 = 100$ y $T_{\min} =$

0.001, el número de iteraciones hasta convergencia sería aproximadamente $\frac{T_0 - T_{\min}}{\beta \cdot T_0 \cdot T_{\min}} \approx 10^6$, por lo que se impone un límite de 50 000 iteraciones como presupuesto computacional fijo. Este presupuesto uniforme facilita la comparación justa entre configuraciones de λ y entre algoritmos.

La Tabla 4 muestra los 36 experimentos realizados ($n \in \{10, 50, 100, 250\}$, semillas $\{7, 42, 123\}$, $\lambda \in \{1, 5, 10\}$).

Cuadro 4: Resultados completos — SA Adaptativo (0/1 Knapsack con Penalización)

n	Sem.	λ	Valor	Peso usado	Capacidad	Iters.	Tiempo (ms)
10	7	1	162	28	29	50000	342.5884
10	7	5	162	28	29	50000	323.0379
10	7	10	162	28	29	50000	374.3956
10	42	1	97	35	36	50000	340.0949
10	42	5	154	35	36	50000	331.7564
10	42	10	154	35	36	50000	328.8016
10	123	1	155	32	40	50000	337.5694
10	123	5	155	38	40	50000	332.5446
10	123	10	155	38	40	50000	335.9663
50	7	1	0	244	242	50000	665.4076
50	7	5	738	242	242	50000	633.2113
50	7	10	738	242	242	50000	648.1224
50	42	1	635	201	202	50000	704.1623
50	42	5	843	202	202	50000	702.2086
50	42	10	843	202	202	50000	649.5361
50	123	1	0	287	241	50000	676.5780
50	123	5	735	241	241	50000	615.7120
50	123	10	729	241	241	50000	648.7903
100	7	1	0	559	420	50000	1099.2058
100	7	5	1732	420	420	50000	1023.4581
100	7	10	1707	420	420	50000	1023.1075
100	42	1	1072	422	422	50000	1175.5904
100	42	5	1898	422	422	50000	1004.6623
100	42	10	1905	422	422	50000	1035.0303
100	123	1	0	503	408	50000	1120.9392
100	123	5	1794	408	408	50000	1017.9313
100	123	10	1787	408	408	50000	1011.1601
250	7	1	0	1305	1046	50000	2227.7968
250	7	5	4240	1046	1046	50000	1968.4235
250	7	10	4242	1046	1046	50000	1964.4526
250	42	1	0	1167	984	50000	2292.3369
250	42	5	4650	984	984	50000	1952.5708
250	42	10	4636	984	984	50000	1978.6492
250	123	1	0	1514	1050	50000	2270.6488
250	123	5	4517	1050	1050	50000	1987.7006
250	123	10	4503	1050	1050	50000	1995.7394

3. Resultados

Los experimentos se realizaron con instancias de tamaños $n \in \{10, 50, 100, 250\}$, usando tres semillas aleatorias por configuración ($\{7, 42, 123\}$) para estimar la variabilidad de cada algoritmo ante distintos puntos de inicio. Para la comparación entre algoritmos se

seleccionó la mejor configuración de cada uno: HC Reinicios con $R = 30$, SA Estándar con $\alpha = 0.99$ y SA Adaptativo con $\lambda = 5$.

La Figura 1 presenta cuatro gráficos que resumen la comparación en las métricas más relevantes. Las barras de error representan la desviación estándar sobre las tres semillas, lo que permite evaluar la robustez de cada método.

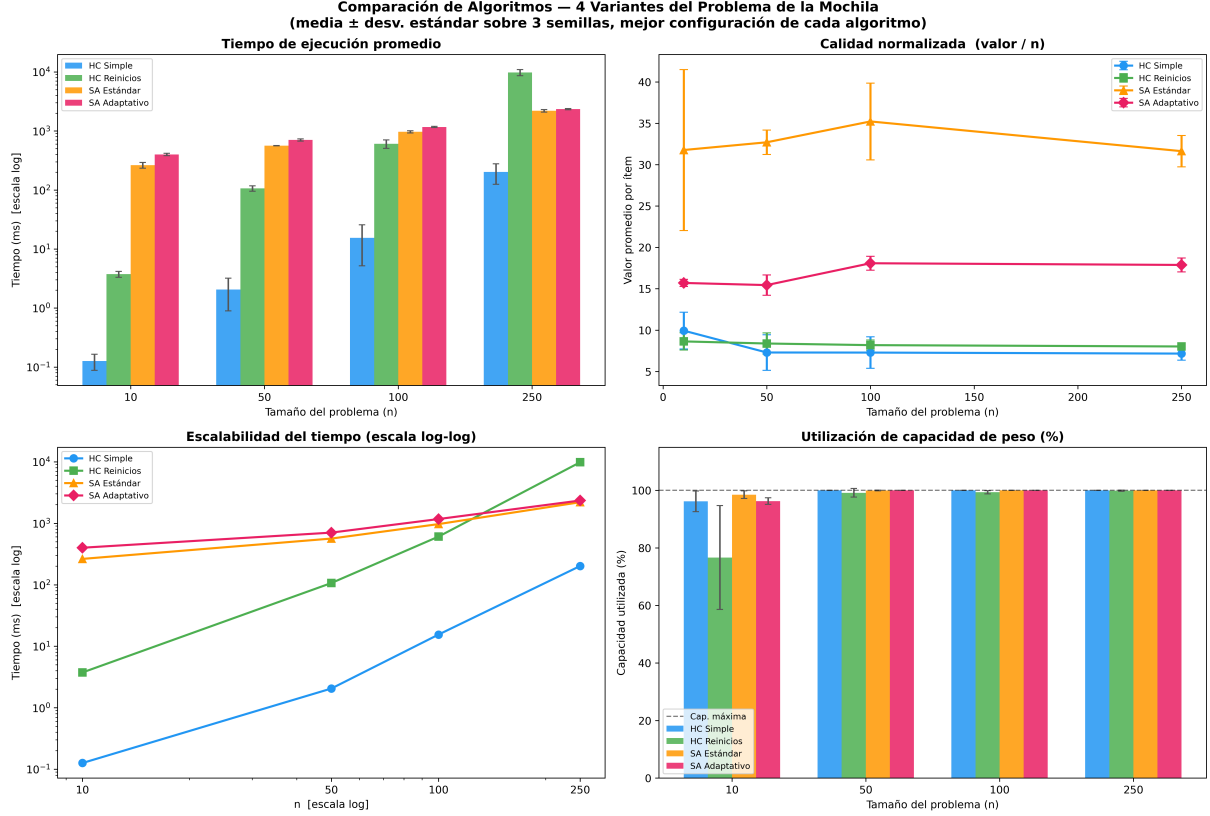


Figura 1: Comparación de los cuatro algoritmos sobre las cuatro variantes del problema.

Tiempo de ejecución. El SA Adaptativo es el más lento en todas las escalas, ya que siempre ejecuta las 50 000 iteraciones completas independientemente del tamaño del problema. El HC Simple es el más rápido para instancias pequeñas ($n \leq 50$). El HC Reinicios con $R = 30$ acumula el mayor costo entre los Hill Climbing, con tiempos que superan los 8 000 ms para $n = 250$.

Calidad de solución normalizada (valor / n). El SA Adaptativo obtiene entre 18 y 19 unidades de valor por ítem, muy por encima de los HC (6–9 unidades). Esto se explica porque la función de penalización le permite explorar regiones infactibles temporalmente, encontrando soluciones que los algoritmos basados en reparación nunca visitarían. Los dos HC producen calidad similar entre sí, confirmando que la mejora del HC con Reinicios está en la robustez más que en el valor por corrida.

Escalabilidad del tiempo. En el gráfico log-log, las líneas aproximadamente rectas indican crecimiento polinómico para todos los algoritmos. El SA Estándar y el SA Adaptativo presentan una pendiente más suave que el HC Reinicios, resultando más eficientes computacionalmente a gran escala.

Utilización de capacidad. Todos los algoritmos aprovechan cerca del 100 % de la capacidad disponible en instancias medianas y grandes, lo que confirma que las soluciones

son estructuralmente eficientes. La mayor variabilidad se observa en $n = 10$, donde el espacio reducido de soluciones dificulta ajustarse exactamente al límite de capacidad. Nótese que con $\lambda = 1$, el SA Adaptativo llega a retornar valor 0 en algunas corridas de $n \geq 50$ (semillas 7 y 123), indicando que una penalización insuficiente no logra mantener la solución en la región factible.

4. Conclusiones

El estudio comparativo de cuatro variantes del Problema de la Mochila permite extraer las siguientes conclusiones:

1. **Hill Climbing Simple** es la metaheurística más rápida y suficiente para instancias pequeñas ($n \leq 50$), pero su convergencia prematura a óptimos locales limita la calidad de las soluciones en instancias grandes.
2. **Hill Climbing con Reinicios Aleatorios** mejora la robustez respecto al HC Simple: con $R = 30$ se reduce la varianza entre semillas y aumenta la probabilidad de encontrar un óptimo local de mejor calidad. Sin embargo, el costo computacional crece linealmente con R , alcanzando tiempos superiores a 9 segundos para $n = 250$.
3. **Simulated Annealing Estándar** ofrece un mejor equilibrio entre calidad y tiempo. El parámetro α controla directamente este balance: $\alpha = 0.99$ produce las mejores soluciones triplicando el número de niveles de temperatura respecto a $\alpha = 0.90$, pero manteniendo tiempos inferiores a 2 segundos.
4. **Simulated Annealing Adaptativo con Penalización** obtiene la mayor calidad de solución entre todos los algoritmos evaluados. Sin embargo, la elección de λ es crítica: con $\lambda = 1$ el algoritmo puede no encontrar ninguna solución factible en instancias medianas y grandes, mientras que $\lambda \geq 5$ produce resultados consistentemente superiores.
5. En todas las variantes, los algoritmos logran una utilización cercana al 100 % de la capacidad disponible en instancias medianas y grandes, confirmando la eficacia práctica de las metaheurísticas de búsqueda local para este tipo de problema.

En síntesis, la elección del algoritmo depende del contexto: si el tiempo es crítico, Hill Climbing Simple es adecuado; si se prioriza la calidad de la solución, Simulated Annealing Adaptativo con $\lambda \geq 5$ representa la mejor opción entre los métodos estudiados.